CEN 4930 Assignment 2 — Multi-Agent and MCP Integration

Nolan Stilwell

Section 1: Part A — MCP Design Rationale

I built the policy tool because the document is a local file I wrote. Gulf Cypress University is fictional, so no hosted MCP server can return CMS-4, DAR-1, and ACN-2. Putting those names in a prompt would not make them a tool. Writing the server meant choosing stdio, the tool schema, and what happens when the process dies. A pre-built server would have hidden those choices.

lookup_major_change_policy takes one string, query. The Policy Agent already holds a JSON handoff, so it can ask in one sentence for the process, deadline, forms, GPA minimum, and prerequisites. The tool always returns a FACTS block from the labeled lines at the top of policy_doc_Stilwell.txt, then up to three narrative sections scored against the query. The facts are the contract. A weak query can miss the Deadlines heading and still cannot drop the deadline, because those lines are not scored. I called the tool through the Agents SDK over stdio. The Policy Agent's filtered list contains only that tool, even though the same process also serves the Part B hazard tool.

If connect() fails, the script prints MCP OFFLINE and the handoff says not to invent form numbers. The advisor can still write a plan, but the policy section has to say the deadline was not confirmed. The record check does not use the server, so a dead process cannot also erase the GPA.

Section 2: Part A — Multi-Agent Architecture Rationale

The transcript and the registrar's rule are different kinds of claims. The Eligibility Agent may read only student_record_Stilwell.txt and checks GPA, posted prerequisites, standing, and holds. The Policy Agent may call only the MCP tool and does not recompute the GPA. The Advisor Agent is the only one that speaks to the student, and the output has to land on four headings. If the first two jobs shared an agent, a number from the advising file could be repeated as if the registrar had published it.

The first handoff is JSON. The eligibility object carries the name, cumulative GPA, prerequisite check, status, and the note that MAC 2311 is still in progress. JSON keeps 3.41 from being softened to "about a 3.4." The second handoff is JSON too: eligibility_json and policy_text side by side, plus the original question. The advisor has to know which lines came from the file and which came from the server.

On "Switch major now", the Eligibility Agent's guardrail trips before any tool call. The student hears: "Please send a full advising question of at least five words about a major, a course plan, or an academic policy. I cannot start an eligibility check from a fragment." A longer pizza question trips the same guardrail because it is outside advising, business, and technology. A plan with blank sections trips the output guardrail. A plan built from the record does not. The advisor retries once, then a second thin plan is flagged.

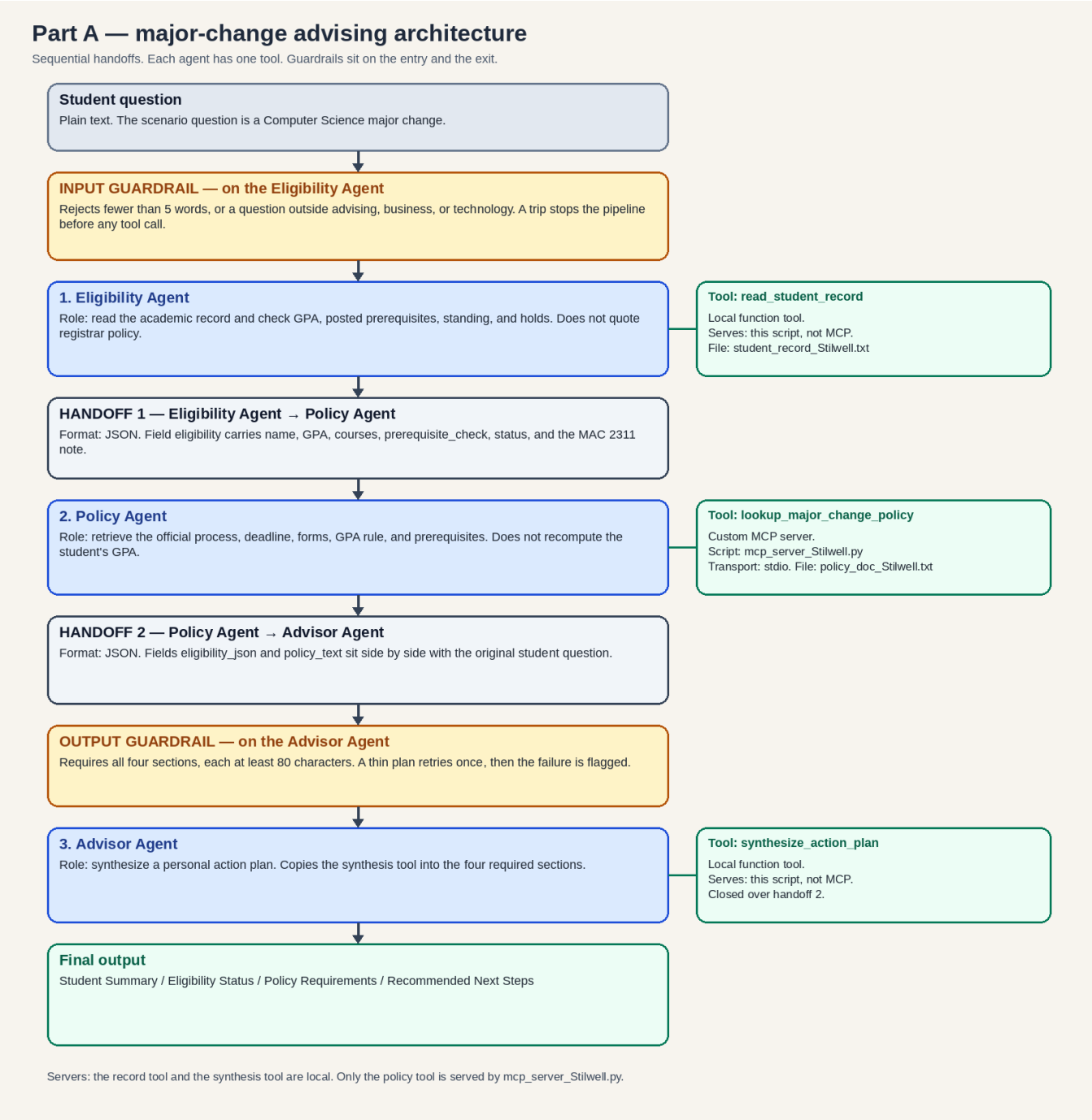


Figure 1. Part A flow. Input guardrail, three agents, two JSON handoffs, output guardrail, and the server for each tool.

Section 3: Part B — Scenario Rationale

I paddle at Bonita Beach before a 9:30 class in Holmes Hall. The car has to be parked by 9:10, my shoulder has been sore, and the tower is empty until 8:00. I wanted a go or no-go with a time on it.

The Session Log Agent only reads session_log_Stilwell.txt: out of the water at 7:35, no overhead takeoffs, no buddy. The Hazard Agent only calls lookup_beach_hazards, and the server is filtered so it cannot see the university policy tool. The Skipper Agent calls build_dawn_patrol_call and cannot invent a new exit time. Drop the hazard agent and the personal log can ignore a yellow flag. Drop the log agent and a beach note can

ignore the class.

The choice that differs most from Part A is the answer. Part A must produce four sections and retries when they are thin. Part B returns GO or NO-GO, and the two tests are supposed to disagree. Waiting for the lifeguard at 8:00 fails the 45-minute rule once the drive is included, so that question is NO-GO even though the early session is GO. The question has to ride in the handoff or the skipper cannot tell those cases apart.

Section 4: What Broke + Model Stack Note

The course samples import FastMCP from mcp.server.fastmcp. On mcp 2 that import raises ModuleNotFoundError and says the class moved to MCPServer. The server tries FastMCP, then falls back. After that, a stdio call to lookup_major_change_policy returned the CMS-4 lines. I also called my output-guardrail decorator directly. It is not a function. The run died with TypeError: 'OutputGuardrail' object is not callable, and the finally block stopped the policy server. I call OutputGuardrail.run now. The thin plan trips. The full plan does not.

The model never finished a handoff here. After the guardrail allowed the Computer Science question, Runner.run entered the Eligibility Agent and raised OpenAIError: Missing credentials. I did not fake a completion. I also did not ask the model to hand off by obeying "Always hand off." The script passes a JSON payload into the next run, and the synthesis tool is closed over that payload.

AI Disclosure

I used Cursor's cloud agent, running Grok 4.7, to draft the Python files, the fictional data files, the architecture diagram, and this report. I kept the Agents SDK patterns from the chapter 3 and chapter 4 samples in the course repository. The eligibility rules, the policy text, and the dawn-patrol scenario were written for this submission. I checked them by running the guardrails, the file parsers, and a live stdio call into mcp_server_Stilwell.py. No live model completion was available here because OPENAI_API_KEY was not set.